ChatGPT

Dynamic Role and Privilege System for Grace Connect

Executive summary

I first attempted connector discovery through the enabled `api_tool`, but no GitHub function surfaced in the current in-scope tool list, so the repository scan below uses the public GitHub repository directly. The public repository is mixed-content: the root README describes a YouTube automation project, while the Grace Connect application relevant to this request lives under `Documents/my_church_app/graceconnect_app`. That nested app is a Flutter web application, built for static deployment on Render, and it initializes both Supabase and Firebase. The code comments and services show a split backend model: Supabase is used for auth and profile data, while Firestore still powers several non-auth features. [1]

The current authorization model is inconsistent and only partially normalized. Regular signup writes `roles: ['Member']` into the `users` table, church signup writes `roles: ['Admin', 'Pastor']`, `user_profile.dart` still treats roles as free-form strings and exposes legacy helpers like `isPastor`, `isAdmin`, and capability getters, and the current login flow routes the user to `/admin_dashboard` after loading the profile instead of making a privilege-based landing decision. This is a strong signal that Grace Connect needs a real authorization subsystem rather than more hard-coded role checks. [2]

The best-fit implementation for this codebase is a hybrid design: normalized RBAC for reusable roles and privileges, plus assignment-level privilege overrides so the UI can ask, at assignment time, which privileges to activate for that specific user. In practice, that means roles become templates, role privileges become default or optional grants, and the assignment modal stores the delta between the template and the chosen grant set. Supabase Edge Functions are the cleanest place for the REST API because they are authenticated TypeScript HTTP endpoints and the Supabase gateway can validate JWTs before the function runs. RLS should protect any browser-exposed tables or views, and all privileged writes should be validated server-side on every request. [3]

The main architectural caveat is Firestore. Firebase's security rules rely on `request.auth`, and the Firebase docs explicitly tie that to Firebase Authentication or Identity Platform. Grace Connect's `main.dart` explicitly says auth state is driven only by Supabase, not Firebase Auth. That means UI-only privilege checks are not enough for any Firestore mutation that matters. For privileged Firestore actions such as schedule management, stream configuration, or church updates, the safe path is to proxy those writes through server-side endpoints or, as a larger future change, add a Firebase-auth bridge with custom claims. [4]

Repository findings and assumptions

The relevant application surface appears to be the Flutter app under `Documents/my_church_app/graceconnect_app`. The repo tree positively exposes `models`, `screens`, `services`, and

`main.dart`, while `main.dart` also imports additional directories such as `providers`, `theme`, `widgets`, dashboard variants, and admin screens. Because those imported paths were not all directly visible in the public tree scan, the instructions below are written to be implementable against the scanned files plus the import graph inferred from `main.dart`; an AI developer should verify the local working tree before patching unresolved imports. [5]

The observed current state is summarized below.

Observation	Why it matters
The root repository is not purely Grace Connect; it also contains unrelated automation assets and a root README about YouTube Shorts automation. [6]	Any implementation should target the nested church app, not the repo root README narrative.
`main.dart` imports dashboards, admin/member management, attendance, donations, finance, events, prayers, analytics, counseling, live streaming, community feed, Bible, study groups, settings, profile, feedback, support, and a developer console. [7]	The privilege catalog should be feature-based, not role-name-based.
Regular signup writes `roles: ['Member']`; church signup writes `roles: ['Admin', 'Pastor']`. [8]	A many-to-many user-role model is required from day one.
`user_profile.dart` still contains hard-coded role helpers and a capability wrapper seeded from role IDs. [9]	Backward compatibility is necessary, but capability checks should move from ad hoc role strings to explicit privilege keys.
The login flow loads the user profile and then navigates to `/admin_dashboard`. [10]	Landing routes and UI affordances must become privilege-aware.
The app uses Supabase for auth/profile tables but still uses Firestore for church schedules, stats, and stream settings. [11]	Authorization enforcement must handle a dual-backend reality.

The design below assumes that `users.uid` is the canonical user identifier, that `users.placeId` and `churches.place_id` are the practical tenant-scoping keys, and that roles may be church-scoped or global. That assumption matches how signup and church creation currently write profiles and church records. It also assumes that the unresolved `UserRoleProvider` is the existing state-management integration point to extend or replace with a more explicit `AuthorizationProvider`. [12]

Privilege inventory

The feature map below is synthesized from the Flutter route registry in `main.dart`, the current signup and login flows, the user profile model, and `church_service.dart`, which already performs schedule, church, stream, search, and seed operations. Because no dedicated announcements screen was visible in the scan, announcements are modeled as a first-class capability layered onto the existing community and event surfaces. [13]

Feature area	Privilege keys	Description
Profile and account	`profile.read_self`, `profile.update_self`, `profile.read_any`, `profile.update_any`	Own profile editing; elevated profile access for admins/ staff.
Members and user management	`members.read`, `members.create`, `members.update`, `members.remove`, `members.invite`	Member directory, membership maintenance, invites, approval workflows.
Roles and privileges	`roles.read`, `roles.create`, `roles.update`, `roles.archive`, `roles.assign`, `privileges.read`	Dynamic role authoring and assignment.
Attendance	`attendance.read`, `attendance.mark`, `attendance.update`, `attendance.delete`, `attendance.export`	Marking attendance and maintaining attendance records.
Donations	`donations.read`, `donations.record`, `donations.update`, `donations.delete`	Donation entry and stewardship workflows.
Finance dashboard	`finance.dashboard.read`, `finance.export`, `finance.settings.update`	Access to finance reporting and settings.
Events	`events.read`, `events.create`, `events.update`, `events.delete`, `events.publish`	Event authoring and publishing.
Service schedules	`services.schedule.read`, `services.schedule.create`, `services.schedule.update`, `services.schedule.delete`	Church schedule CRUD surfaced by `church_service.dart`.
Prayer	`prayers.read`, `prayers.create`, `prayers.update`, `prayers.delete`, `prayers.assign`	Prayer requests plus any staff-led assignment workflow.
Analytics	`analytics.read`	Read-only access to analytics and church stats.
Counseling	`counseling.read`, `counseling.manage`	Sensitive pastoral or support workflows.
Live streaming	`stream.read`, `stream.manage`	Read stream status vs update stream URL and live flags.
Community feed	`community.read`, `community.post`, `community.update`, `community.delete`, `community.moderate`	Community posting and moderation.

Feature area	Privilege keys	Description
Announcements	`announcements.post`, `announcements.publish`, `announcements.delete`	A scoped content type for church-wide announcements, likely implemented inside community/events.
Bible	`bible.read`, `bible.settings.update`	Bible view plus any Bible-related settings surface.
Study groups	`study_groups.read`, `study_groups.create`, `study_groups.update`, `study_groups.delete`	Group creation and management.
Settings	`settings.account.update`, `settings.privacy.update`, `settings.notifications.update`, `settings.attendance.update`, `settings.community.update`, `settings.church_admin.update`, `settings.finance.update`, `settings.app.update`	Split settings by surface so access can be delegated safely.
Feedback and support	`feedback.read`, `feedback.respond`, `support.manage`	Staff-side handling of inbound support/feedback.
Churches and registry	`churches.read`, `churches.update`, `churches.register_manual`	Church search, editing, and manual registration.
Developer console	`developer_console.access`, `churches.seed`	High-risk internal tools like seeding initial churches.

A practical seed set of roles should remain data-driven, not hard-coded, but these starter templates fit the repo scan and the user's requested examples.

Role slug	Default privileges	Optional privileges	Notes
`guest`	`events.read`, `bible.read`, `stream.read`, `community.read`	none	Minimal read-only presence.
`member`	Guest defaults plus `profile.update_self`, `attendance.mark`, `prayers.create`, `community.post`, `study_groups.read`	`events.create`	Safe standard member role.

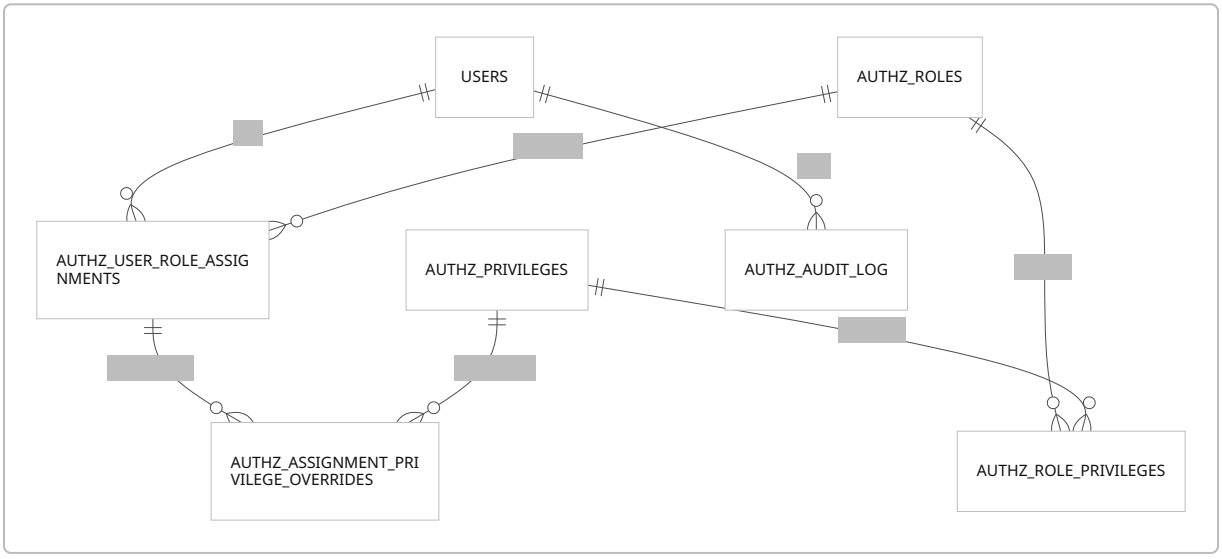
Role slug	Default privileges	Optional privileges	Notes
`moderator`	Member defaults plus `community.moderate`, `community.update`, `community.delete`	`announcements.publish`	Good for feed moderation without admin powers.
`announcer`	`community.read`, `announcements.post`, `announcements.publish`, `events.read`, `events.create`, `events.update`	`stream.manage`	Best match for the requirement that assignments may trigger announcement privileges.
`pastor`	Member defaults plus `prayers.assign`, `counseling.read`, `counseling.manage`, `members.read`, `analytics.read`	`events.publish`	Matches current `Pastor` seed concept.
`finance_manager`	`donations.*`, `finance.dashboard.read`, `finance.export`	`analytics.read`, `settings.finance.update`	Avoid broad admin rights.
`admin`	Broad access across members, roles, settings, analytics, schedules, stream, community moderation	`developer_console.access` only if separately approved	Church-scoped administrator.
`platform_developer`	`developer_console.access`, `churches.seed`, `roles.read`	optional narrow debugging grants	Prefer this global role over normal church-scoped `developer`; the repo already carries an `isDeveloper` flag in user profiles. [14]

The key design choice is that a role is a reusable privilege envelope, not the final grant artifact. Assignment-time selection is what turns “Admin with defaults” into “Admin but not finance settings,” or “Announcer plus stream management.” That keeps RBAC manageable while using ABAC-style context such as church scope, active account state, and validity windows to activate or suppress privileges at runtime. [15]

Data model and backend design

Schema

Grace Connect currently stores roles as raw arrays on `users`, but NIST's RBAC model is built on explicit user-role and role-permission relations, and ABAC guidance is a good fit for adding contextual checks such as `placeId`, account status, and time windows. The schema below keeps that separation clean: reusable roles, reusable privileges, role-level defaults and optionals, per-user role assignments, per-assignment overrides, and an audit log. [16]



A compact schema summary:

Table	Purpose	Key columns
`authz.privileges`	Master privilege catalog	`key`, `category`, `description`, `is_system`
`authz.roles`	Reusable role templates	`place_id`, `slug`, `name`, `is_system`, `archived_at`
`authz.role_privileges`	Role envelope	`role_id`, `privilege_id`, `grant_mode` (`default` or `optional`)
`authz.user_role_assignments`	Actual user-role grant	`user_id`, `role_id`, `place_id`, `assigned_by`, `starts_at`, `ends_at`, `revoked_at`

Table	Purpose	Key columns
`authz.assignment_privilege_overrides`	Assignment-specific deltas	`assignment_id`, `privilege_id`, `effect` (`allow` or `deny`)
`authz.audit_log`	Change history	`actor_user_id`, `target_user_id`, `entity_type`, `before_state`, `after_state`

A concrete migration skeleton for Supabase/Postgres looks like this. It is designed to coexist with the existing `public.users` table and `placeId` church scoping already visible in the repo. Supabase recommends managing schema changes as SQL migrations through the CLI rather than editing the remote database directly. [17]

```sql
create schema if not exists authz;

create extension if not exists pgcrypto;

create type authz.effect as enum ('allow', 'deny');
create type authz.grant_mode as enum ('default', 'optional');

create table if not exists authz.privileges (
  id uuid primary key default gen_random_uuid(),
  key text not null unique check (key ~ '^[a-z0-9._]+$'),
  category text not null,
  title text not null,
  description text not null,
  is_system boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table if not exists authz.roles (
  id uuid primary key default gen_random_uuid(),
  place_id text null, -- null = global/system role
  slug text not null check (slug ~ '^[a-z0-9_]+$'),
  name text not null,
  description text not null default '',
  is_system boolean not null default false,
  created_by text null references public.users(uid),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  archived_at timestamptz null
);
```

```
create unique index if not exists authz_roles_scope_slug_idx
  on authz.roles ((coalesce(place_id, '__global__')), slug);

create table if not exists authz.role_privileges (
  role_id uuid not null references authz.roles(id) on delete cascade,
  privilege_id uuid not null references authz.privileges(id) on delete cascade,
  grant_mode authz.grant_mode not null default 'default',
  primary key (role_id, privilege_id)
);

create table if not exists authz.user_role_assignments (
  id uuid primary key default gen_random_uuid(),
  user_id text not null references public.users(uid) on delete cascade,
  role_id uuid not null references authz.roles(id) on delete cascade,
  place_id text not null,
  assigned_by text null references public.users(uid),
  reason text null,
  starts_at timestamptz not null default now(),
  ends_at timestamptz null,
  revoked_at timestamptz null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create unique index if not exists authz_active_user_role_idx
  on authz.user_role_assignments (user_id, role_id, place_id)
  where revoked_at is null and ends_at is null;

create table if not exists authz.assignment_privilege_overrides (
  assignment_id uuid not null references authz.user_role_assignments(id) on
delete cascade,
  privilege_id uuid not null references authz.privileges(id) on delete cascade,
  effect authz.effect not null,
  created_at timestamptz not null default now(),
  primary key (assignment_id, privilege_id)
);

create table if not exists authz.audit_log (
  id bigint generated always as identity primary key,
  actor_user_id text null references public.users(uid),
  target_user_id text null references public.users(uid),
  place_id text null,
  action text not null,
  entity_type text not null,
  entity_id text not null,
  before_state jsonb null,
  after_state jsonb null,
```

```
  created_at timestamptz not null default now()
);

alter table public.users
  add column if not exists authz_version bigint not null default 0;
```

For backward compatibility, I recommend a transition trigger that mirrors legacy `users.roles` into the new tables until the UI stops writing raw role arrays. Because the existing column type is not visible in the public scan, the safest backfill pattern is to normalize through `to_jsonb(users.roles)` and then expand the array. [18]

```
with legacy_roles as (
  select
    u.uid as user_id,
    u."placeId" as place_id,
    regexp_replace(lower(trim(role_name)), '[^a-z0-9]+', '_', 'g') as role_slug,
    initcap(trim(role_name)) as role_name
  from public.users u
  cross join lateral jsonb_array_elements_text(
    coalesce(to_jsonb(u.roles), '[]'::jsonb)
  ) as role_name
)
insert into authz.roles (place_id, slug, name, is_system)
select distinct place_id, role_slug, role_name, true
from legacy_roles
on conflict do nothing;
```

APIs and authorization

For this repo, the cleanest “backend” is a small REST surface implemented as Supabase Edge Functions. Edge Functions are server-side TypeScript functions, the gateway handles auth headers and JWT validation, and they are explicitly suitable for authenticated HTTP endpoints. Any authz data exposed directly to the browser should still have RLS enabled, because Supabase requires RLS on exposed schema tables and views. [19]

A minimal endpoint surface:

Method	Path	Purpose	Required privilege
`GET`	`/api/v1/privileges`	List privilege catalog grouped by category	`privileges.read` or `roles.assign`
`GET`	`/api/v1/roles`	List roles in current church plus global roles	`roles.read`

Method	Path	Purpose	Required privilege
`POST`	`/api/v1/roles`	Create role and attach default/ optional privileges	`roles.create`
`PATCH`	`/api/v1/roles/:roleId`	Update role metadata and privilege envelope	`roles.update`
`POST`	`/api/v1/users/:userId/role-assignments`	Assign role to user, prompting selected privileges	`roles.assign`
`PATCH`	`/api/v1/users/:userId/role-assignments/:assignmentId`	Edit selected privileges or timing window	`roles.assign`
`DELETE`	`/api/v1/users/:userId/role-assignments/:assignmentId`	Revoke assignment	`roles.assign`
`GET`	`/api/v1/users/:userId/effective-privileges`	View another user's effective privileges	`roles.read`
`GET`	`/api/v1/me/effective-privileges`	Load current user privilege set for UI gating	authenticated self
`POST`	`/api/v1/authorize/check`	Optional single-check helper for client convenience	authenticated self or backend only

A role-create payload should let the form author specify which privileges are defaults and which are optional:

```
{
  "placeId": "church_1717185000000_abc123",
  "slug": "announcer",
  "name": "Announcer",
  "description": "Publishes church announcements and coordinates event
visibility.",
  "privileges": [
    { "key": "community.read", "grantMode": "default" },
    { "key": "announcements.post", "grantMode": "default" },
    { "key": "announcements.publish", "grantMode": "default" },
    { "key": "events.read", "grantMode": "default" },
    { "key": "events.update", "grantMode": "optional" },
    { "key": "stream.manage", "grantMode": "optional" }
```

```
  ]
}
```

The assignment endpoint is where your required prompt happens. The modal shows role defaults preselected, optional privileges unchecked, and the POST body submits the final selected set. The backend then computes overrides: unselected defaults become `deny` rows; selected optional privileges become `allow` rows. That preserves reusable role templates while making assignment-time selection explicit.

```
{
  "placeId": "church_1717185000000_abc123",
  "roleSlug": "announcer",
  "selectedPrivilegeKeys": [
    "community.read",
    "announcements.post",
    "announcements.publish",
    "events.read",
    "events.update"
  ],
  "reason": "Sunday communications team"
}
```

```
{
  "assignmentId": "3e2d5f4f-f16d-4d7c-b5e8-6dc2eac7d2cc",
  "userId": "4df6cb0b-3f95-4e3c-84b6-cd8b0b3f0d71",
  "role": {
    "slug": "announcer",
    "name": "Announcer"
  },
  "placeId": "church_1717185000000_abc123",
  "overrides": [
    { "key": "events.update", "effect": "allow" }
  ],
  "effectivePrivilegeKeys": [
    "announcements.post",
    "announcements.publish",
    "community.read",
    "events.read",
    "events.update"
  ],
  "authzVersion": 12
}
```

The middleware rule should be stricter than “actor has `roles.assign`.” The actor should also be prevented from granting privileges they do not possess, unless there is an explicit delegation privilege such

as `roles.delegate_any`. That is the cleanest way to prevent privilege escalation while honoring least privilege and deny-by-default guidance. [20]

```
// Example for a Supabase Edge Function or small Node service
type AuthzContext = {
  actorUserId: string;
  placeId: string;
  privileges: Set<string>;
};

export async function buildAuthzContext(req: Request): Promise<AuthzContext> {
  const authHeader = req.headers.get('Authorization') ?? '';
  const token = authHeader.replace(/^Bearer\s+/i, '');
  if (!token) throw new Response(JSON.stringify({ error: 'missing_token' }), {
status: 401 });

  // service-role client; validate the caller JWT
  const admin = createAdminSupabaseClient();
  const { data, error } = await admin.auth.getUser(token);
  if (error || !data.user) {
    throw new Response(JSON.stringify({ error: 'invalid_token' }), { status:
401 });
  }

  const actorUserId = data.user.id;
  const placeId = req.headers.get('X-Place-Id') ?? '';

  const { data: rows } = await admin.rpc('authz_effective_privilege_keys', {
    p_user_id: actorUserId,
    p_place_id: placeId,
  });

  return {
    actorUserId,
    placeId,
    privileges: new Set((rows ?? []).map((r: { privilege_key: string }) =>
r.privilege_key)),
  };
}

export function requirePrivilege(ctx: AuthzContext, needed: string) {
  if (!ctx.privileges.has(needed)) {
    throw new Response(JSON.stringify({ error: 'forbidden', needed }), {
status: 403 });
  }
}
```

```ts
export function assertGrantSubset(
  ctx: AuthzContext,
  requested: string[],
  roleEnvelope: Set<string>
) {
  for (const key of requested) {
    if (!roleEnvelope.has(key)) {
      throw new Response(JSON.stringify({ error: 'invalid_privilege_for_role',
key }), { status: 400 });
    }
    if (!(ctx.privileges.has(key) || ctx.privileges.has('roles.delegate_any')))
{
      throw new Response(JSON.stringify({ error:
'cannot_grant_privilege_you_do_not_have', key }), { status: 403 });
    }
  }
}
```

Effective privilege SQL

The query below computes effective privileges for a user across multiple active assignments, with assignment-level allow and deny overrides. Default privileges come from the role template; `allow` overrides opt the assignment into optional privileges; `deny` overrides opt the assignment out of default privileges. Since effective access is a union across assignments, a deny only cancels that privilege for the specific assignment it belongs to. That keeps “Moderator denies event publishing” from accidentally stripping event publishing granted by a separate “Admin” assignment. [15]

```sql
create or replace function authz.authz_effective_privilege_keys(
  p_user_id text,
  p_place_id text
)
returns table (privilege_key text)
language sql
stable
as $$
with active_assignments as (
  select id, role_id
  from authz.user_role_assignments
  where user_id = p_user_id
    and place_id = p_place_id
    and revoked_at is null
    and (starts_at is null or starts_at <= now())
    and (ends_at is null or ends_at > now())
),
default_role_grants as (
  select a.id as assignment_id, rp.privilege_id
```

```
  from active_assignments a
  join authz.role_privileges rp
    on rp.role_id = a.role_id
   and rp.grant_mode = 'default'
),
override_allows as (
  select o.assignment_id, o.privilege_id
  from authz.assignment_privilege_overrides o
  join active_assignments a on a.id = o.assignment_id
  where o.effect = 'allow'
),
override_denies as (
  select o.assignment_id, o.privilege_id
  from authz.assignment_privilege_overrides o
  join active_assignments a on a.id = o.assignment_id
  where o.effect = 'deny'
),
assignment_net as (
  select d.assignment_id, d.privilege_id
  from default_role_grants d
  left join override_denies x
    on x.assignment_id = d.assignment_id
   and x.privilege_id = d.privilege_id
  where x.privilege_id is null

  union

  select assignment_id, privilege_id
  from override_allows
)
select distinct p.key as privilege_key
from assignment_net n
join authz.privileges p on p.id = n.privilege_id
order by p.key;
$$;
```

A single-check query for middleware or tests is then simple:

```
select exists (
  select 1
  from authz.authz_effective_privilege_keys(
    '4df6cb0b-3f95-4e3c-84b6-cd8b0b3f0d71',
    'church_1717185000000_abc123'
  )
```

```
  where privilege_key = 'roles.assign'
) as can_assign_roles;
```

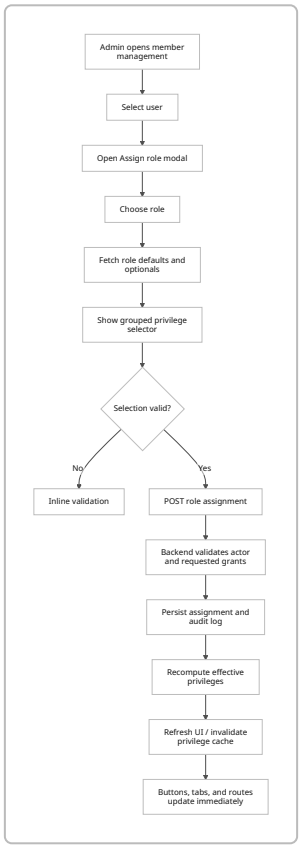
Finally, because Grace Connect still performs client-side Firestore mutations in `church_service.dart`, every privileged Firestore write should either move behind these same endpoints or be protected by a Firebase-auth bridge. Firebase's docs are clear that Firestore rules rely on `request.auth`, and custom claims must be set from privileged server code and are meant for access control, not general profile storage. In the current architecture, backend proxying is the lower-friction path. [21]

Frontend flow and repository integration

The actual Grace Connect app is Flutter, not React, so the production integration should use Flutter widgets and Provider-style state. The React-style contracts below are included only because you asked for a framework-neutral REST/UI spec; the concrete repo integration notes that follow are Flutter-first because that is what the scanned project uses. [22]

UX flow

The assignment UX should work like this:



The role-creation form should collect role name, slug, description, and a categorized privilege matrix. Each privilege row should let the admin choose whether the privilege is part of the role by default or only

optionally assignable later. The role-assignment modal should then show the chosen role's defaults preselected and its optionals unselected, with category groupings like Members, Community, Finance, Settings, Developer, and so on. That creates the exact behavior you requested: every time a role is assigned, the UI prompts which privileges to enable for that user. The modal should also show a concise summary panel such as "5 default privileges, 2 optional selected, 1 denied." [20]

A portable component contract might look like this:

```
type Privilege = {
  key: string;
  category: string;
  title: string;
  description: string;
  grantMode: 'default' | 'optional';
};

type RoleEditorProps = {
  initialRole?: {
    id?: string;
    slug: string;
    name: string;
    description: string;
    privileges: Array<{ key: string; grantMode: 'default' | 'optional' }>;
  };
  privileges: Privilege[];
  onSubmit: (payload: {
    slug: string;
    name: string;
    description: string;
    privileges: Array<{ key: string; grantMode: 'default' | 'optional' }>;
  }) => Promise<void>;
};

type AssignRoleModalProps = {
  open: boolean;
  userId: string;
  placeId: string;
  role: {
    id: string;
    slug: string;
    name: string;
    privileges: Privilege[];
  } | null;
  onClose: () => void;
  onAssigned: (result: { effectivePrivilegeKeys: string[]; authzVersion:
```

```
number }) => void;
};
```

And the assignment interaction itself should be simple and explicit:

```
function AssignRoleModal(props: AssignRoleModalProps) {
  const [selected, setSelected] = React.useState<Set<string>>(new Set());
  const [saving, setSaving] = React.useState(false);

  React.useEffect(() => {
    if (!props.role) return;
    const defaults = props.role.privileges
      .filter((p) => p.grantMode === 'default')
      .map((p) => p.key);
    setSelected(new Set(defaults));
  }, [props.role]);

  async function submit() {
    if (!props.role) return;
    setSaving(true);
    try {
      const res = await fetch(`/api/v1/users/${props.userId}/role-assignments`,
{
        method: 'POST',
        headers: { 'Content-Type': 'application/json', 'X-Place-Id':
props.placeId },
        body: JSON.stringify({
          placeId: props.placeId,
          roleSlug: props.role.slug,
          selectedPrivilegeKeys: [...selected],
        }),
      });
      const json = await res.json();
      props.onAssigned(json);
      props.onClose();
    } finally {
      setSaving(false);
    }
  }

  return null; // Render grouped checkboxes by category in the real component.
}
```

In Flutter, the same concepts become `RoleManagementScreen`, `RoleEditorDialog`, `PrivilegeSelectorSheet`, `AssignRoleDialog`, and `PrivilegeGate`. Because `main.dart` already uses Provider, the most natural implementation is an `AuthorizationProvider` with a

`can(String key)` method and a `refresh()` method that hydrates `effectivePrivilegeKeys` from `/api/v1/me/effective-privileges`. [7]

Repository integration points

The following file plan is based on the scan plus the import graph visible in `main.dart`.

Path	Scan status	Required change
`Documents/my_church_app/graceconnect_app/lib/main.dart`	Confirmed	Register `AuthorizationProvider`, add role-management routes, and change route guards / landing-route selection to privilege-aware logic. [7]
`.../lib/models/user_profile.dart`	Confirmed	Add `effectivePrivilegeKeys`, `can(key)`, and compatibility mappings from old helpers to new privilege checks. Preserve `roles` temporarily as a legacy mirror. [9]
`.../lib/screens/login screen/login_screen.dart`	Confirmed	Replace hard-coded `/admin_dashboard` navigation with `AuthorizationProvider.defaultRoute()` after privilege hydration. [10]
`.../lib/screens/signup screen/signup_screen.dart`	Confirmed	Keep legacy `roles` write only during transition, then move member-role bootstrap to backend assignment flow or legacy-sync trigger. [23]
`.../lib/screens/signup screen/church_signup_screen.dart`	Confirmed	Same as above, but bootstrap `admin` and `pastor` assignments server-side; stop trusting direct client role writes long-term. [24]
`.../lib/services/church_service.dart`	Confirmed	Move privileged Firestore mutations (`updateChurch`, `createSchedule`, `deleteSchedule`, `updateStreamSettings`, seeding) behind backend authz checks or dedicated admin endpoints. [25]
`.../supabase/migrations/*`	New	Create `authz` schema, tables, indexes, audit log, backfill, and optional legacy-sync trigger.
`.../supabase/functions/authz-api/index.ts`	New	Implement the REST surface and enforce privilege checks.
`.../lib/services/authorization_service.dart`	New	Wrap `/api/v1/me/effective-privileges`, role CRUD, and assignment APIs.

Path	Scan status	Required change
`.../lib/providers/authorization_provider.dart`	New or merged	Cache current privilege set, expose `can()`, `hasAny()`, `refresh()`, and invalidate after role changes.
`.../lib/widgets/privilege_gate.dart`	New	Hide or disable buttons, tiles, routes, and dashboard cards in real time.
`.../lib/providers/user_role_provider.dart`	Imported in code, not directly surfaced in tree scan	Extend or replace with a privilege-based provider; verify local path first. [7]
`.../lib/screens/admin/member_management_screen.dart`	Imported in code, not directly surfaced in tree scan	Mount the Assign Role modal here. Verify local path first. [7]
`.../lib/screens/dashboard/variants/admin_dashboard.dart` and related dashboard files	Imported in code, not directly surfaced in tree scan	Filter dashboard tiles and admin cards through `PrivilegeGate`. Verify local path first. [7]

The most important concrete diff is the login flow. Today it fetches the user profile and routes to the admin dashboard. That should become a privilege-aware path decision.

```
- Navigator.of(context).pushNamedAndRemoveUntil('/admin_dashboard', (route) =>
false);
+ final authz = context.read<AuthorizationProvider>();
+ await authz.refresh();
+ final landing = authz.defaultRoute(); // e.g. /admin_dashboard or /
member_dashboard
+ Navigator.of(context).pushNamedAndRemoveUntil(landing, (route) => false);
```

`user_profile.dart` should also stop being the authorization engine. Keep it as a user profile plus a privilege cache:

```
class UserProfile {
  final List<String> roles; // legacy mirror during rollout
+ final Set<String> effectivePrivilegeKeys;

+ bool can(String key) => effectivePrivilegeKeys.contains(key);

- bool get isAdmin => roles.contains('Admin') || roles.contains('Pastor');
+ bool get isAdmin => can('roles.assign') || can('members.update');

- bool get isPastor => roles.contains('Pastor');
+ bool get isPastor => can('prayers.assign') || can('counseling.manage');
}
```

Finally, `ChurchService` is the clearest current mutation hotspot from the public scan, so it should be treated as the first service to de-clientize. If you change only one service in phase one, change this one, because it already updates churches, schedules, stats, and stream settings. [25]

Testing, security, and rollout

Testing plan

OWASP recommends least privilege, deny-by-default, and validating permissions on every request, plus dedicated unit and integration tests for authorization logic. In Grace Connect, that means testing both effective privilege computation and the UI and service layers that consume it. [20]

Layer	What to test	Example cases	Suggested tooling
SQL / authz core	Role-default + override semantics	Member + Moderator union; deselected default becomes deny; optional selected becomes allow; expired assignment drops privileges	SQL tests, local Supabase reset, fixture data
Edge Function / REST integration	Endpoint authorization and input validation	Actor without `roles.assign` gets 403; actor cannot grant a privilege they do not hold; cross-church assignment rejected	Deno or Node integration tests
Flutter unit / widget	Local gating and state refresh	`PrivilegeGate` hides analytics tile; role assignment modal preselects defaults; dashboard cards refresh after assignment	`flutter_test`
End-to-end	Full admin workflow	Create role, assign role with customized privileges, sign in as assignee, verify buttons/routes/tiles and forbidden API calls	Flutter `integration_test` or Patrol

Layer	What to test	Example cases	Suggested tooling
Regression safety	Backward compatibility during rollout	Existing `users.roles=['Member']` still yields effective member privileges through legacy-sync trigger	Local migration + seed test

Representative test cases:

- **Unit**: a role with defaults `community.read`, `announcements.post` and optional `events.update`; selected set omits `announcements.post` and includes `events.update`; effective result should be `community.read` and `events.update`, not `announcements.post`.
- **Integration**: `POST /api/v1/users/:id/role-assignments` with `selectedPrivilegeKeys` containing `settings.finance.update` by an actor who lacks that privilege must fail with 403.
- **Integration**: two concurrent assignments to the same user and same role must not create duplicate active rows.
- **E2E**: assign `announcer` without `stream.manage`; assignee sees announcements UI but not stream controls.
- **E2E**: revoke `analytics.read`; analytics route becomes inaccessible after privilege refresh.

Security considerations

The security model should follow a few non-negotiables.

Risk	Required control
Privilege escalation during assignment	Actor must have `roles.assign` **and** every privilege they are trying to grant, unless a separate delegation privilege exists.
Broken access control through hidden-only UI	Every backend endpoint and any Firestore proxy must check privileges server-side on every request.
Cross-church data leakage	Every query and assignment must be scoped by `placeId`; a role assignment must only target a role whose scope is global or matches the user's `placeId`.
Firestore bypass	Do not trust UI gating. Proxy sensitive Firestore writes through the authz backend or add a Firebase-auth bridge with custom claims.
Race conditions / duplicate assignments	Use a transaction plus a unique partial index on active assignments; optionally add an advisory lock on `(user_id, place_id)` in the assignment endpoint.
Input tampering	Validate `slug`, privilege keys, time windows, and selection subsets; reject unknown privilege keys.
Audit gaps	Write an append-only audit log for role create/update/archive, role assignment/revoke, and privilege-envelope changes.
Migration risk	Keep legacy `users.roles` during rollout, back it up, and gate new behavior behind a feature flag.

Supabase's RLS guidance matters here: any table or view reachable from the browser should have RLS enabled, and no data is accessible through the API with a publishable key until policies are created. OWASP's guidance is equally clear that authorization must be deny-by-default and validated on every request, not just "most" requests. [26]

Deployment, rollback, and timeline

Supabase's migration workflow is well-suited to this change: create migrations locally, test with `supabase db reset`, and deploy with `supabase db push`. Supabase explicitly warns against making remote schema changes directly through the dashboard because that bypasses migration history and causes sync problems; it also advises coordinating so only one person pushes migrations at a time. [27]

A safe rollout sequence:

Phase	Action	Backward compatibility	Rollback
Schema introduction	Add `authz` tables, `authz_version`, indexes, and seeds for the privilege catalog	No app behavior changes yet	Drop new schema or disable function routing
Legacy sync	Backfill from `users.roles`; optionally add a temporary sync trigger on `users.roles` changes	Existing signup flows keep working	Remove trigger; keep backfilled rows for inspection
Backend API	Deploy Edge Functions for role CRUD, assignment, effective-privilege lookup, and audit logging	Client still uses old roles array until switched	Disable endpoints behind a feature flag
Client hydration	Add `AuthorizationProvider`, `getMyEffectivePrivileges()`, and `PrivilegeGate`	Keep old `roles` helpers alongside new `can()`	Fall back to old role-string UI
Sensitive service hardening	Move `ChurchService` privileged mutations to backend	Existing read flows still work	Temporarily restore client writes if necessary, though not recommended
Cleanup	Stop writing direct `users.roles` from the client and keep it only as read-only compatibility mirror until all old checks are removed	Final compatibility window	Re-enable legacy sync if needed

Suggested commands:

```
supabase migration new add_authz_schema
supabase db reset
supabase db push
```

The implementation timeline below assumes one engineer or one AI-assisted engineer working in a reasonably healthy local tree. Unresolved imports visible in the public scan may push UI work upward by one effort level if the local tree differs materially from the public repo.

Milestone	Deliverable	Effort	Typical elapsed
Authz catalog and schema	Privilege table, roles, assignments, overrides, audit log	Medium	1–2 days
Backfill and legacy compatibility	Backfill script, legacy-sync trigger, `authz_version` increment strategy	Medium	1–2 days
REST backend	Edge Functions, middleware, request validation, audit writes	Medium	2–3 days
Flutter integration	Provider, route gating, dashboard filtering, role-management UI, assignment modal	High	3–5 days
Firestore hardening	Move privileged `ChurchService` mutations behind backend	High	2–4 days
Tests and rollout	Local reset tests, integration tests, E2E checks, deployment docs	Medium	2–3 days

That puts a realistic first production rollout in the **2–3 week** range for one developer, with the fastest useful milestone being the database/API layer plus admin assignment UI. The highest-risk unknown is not the authorization model itself; it is the current split between Supabase auth and Firestore writes. [28]

Sources

- **Shamzbruv/Grace_Connect public repository**, including the root tree, the nested Flutter church app, `main.dart`, login/signup flows, `user_profile.dart`, `church_service.dart`, and Render deployment/build files. [29]
- **Supabase Row Level Security docs**. [30]
- **Supabase Edge Functions docs**. [31]
- **Supabase database migration docs**. [27]
- **Supabase Flutter auth reference** for `signInWithPassword`. [32]
- **Firebase Firestore security rules docs**. [33]
- **Firebase custom claims docs**. [34]
- **NIST RBAC project and standard references**. [35]
- **NIST SP 800-162 ABAC definition**. [36]
- **OWASP Authorization Cheat Sheet**. [20]

1 6 29 GitHub - Shamzbruv/Grace_Connect · GitHub
https://github.com/Shamzbruv/Grace_Connect?utm_source=chatgpt.com

2 8 12 18 23 raw.githubusercontent.com
https://raw.githubusercontent.com/Shamzbruv/Grace_Connect/refs/heads/master/Documents/my_church_app/graceconnect_app/lib/screens/signup%20screen/signup_screen.dart?utm_source=chatgpt.com

3 15 16 35 Role Based Access Control | CSRC
https://csrc.nist.gov/projects/role-based-access-control?utm_source=chatgpt.com

4 11 28 Grace_Connect/Documents/my_church_app/graceconnect_app/lib/main.dart at master · Shamzbruv/Grace_Connect · GitHub
https://github.com/Shamzbruv/Grace_Connect/blob/master/Documents/my_church_app/graceconnect_app/lib/main.dart?utm_source=chatgpt.com

5 Grace_Connect/Documents/my_church_app/graceconnect_app/lib at master · Shamzbruv/Grace_Connect · GitHub
https://github.com/Shamzbruv/Grace_Connect/tree/master/Documents/my_church_app/graceconnect_app/lib?utm_source=chatgpt.com

7 13 raw.githubusercontent.com
https://github.com/Shamzbruv/Grace_Connect/raw/refs/heads/master/Documents/my_church_app/graceconnect_app/lib/main.dart?utm_source=chatgpt.com

9 14 raw.githubusercontent.com
https://github.com/Shamzbruv/Grace_Connect/raw/refs/heads/master/Documents/my_church_app/graceconnect_app/lib/models/user_profile.dart?utm_source=chatgpt.com

10 raw.githubusercontent.com
https://github.com/Shamzbruv/Grace_Connect/raw/refs/heads/master/Documents/my_church_app/graceconnect_app/lib/screens/login%20screen/login_screen.dart?utm_source=chatgpt.com

17 27 Database Migrations | Supabase Docs
https://supabase.com/docs/guides/deployment/database-migrations?utm_source=chatgpt.com

19 31 Edge Functions | Supabase Docs
https://supabase.com/docs/guides/functions?utm_source=chatgpt.com

20 Authorization - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html?utm_source=chatgpt.com

21 25 raw.githubusercontent.com
https://github.com/Shamzbruv/Grace_Connect/raw/refs/heads/master/Documents/my_church_app/graceconnect_app/lib/services/church_service.dart?utm_source=chatgpt.com

22 Grace_Connect/Documents/my_church_app/graceconnect_app/render_build.sh at master · Shamzbruv/Grace_Connect · GitHub
https://github.com/Shamzbruv/Grace_Connect/blob/master/Documents/my_church_app/graceconnect_app/render_build.sh?utm_source=chatgpt.com

24 raw.githubusercontent.com
https://github.com/Shamzbruv/Grace_Connect/raw/refs/heads/master/Documents/my_church_app/graceconnect_app/lib/screens/signup%20screen/church_signup_screen.dart?utm_source=chatgpt.com

26 30 Row Level Security | Supabase Docs
https://supabase.com/docs/guides/database/postgres/row-level-security?utm_source=chatgpt.com

32 Flutter: Sign in a user | Supabase Docs
https://supabase.com/docs/reference/dart/auth-signinwithpassword?utm_source=chatgpt.com

33 Writing conditions for Cloud Firestore Security Rules | Firebase
https://firebase.google.com/docs/firestore/security/rules-conditions?utm_source=chatgpt.com

34 Control Access with Custom Claims and Security Rules | Firebase Authentication
https://firebase.google.com/docs/auth/admin/custom-claims?utm_source=chatgpt.com

36 SP 800-162, Guide to Attribute Based Access Control (ABAC) Definition and Considerations | CSRC
https://csrc.nist.gov/pubs/sp/800/162/upd2/final?utm_source=chatgpt.com